Evaluating the Robustness of LLM Safety Filters against Paraphrased OS Instructions

Literature Review

Pragati Anant Tamboli, Tanishq Dahiya

A00051472 , A00049419

School of Computing

Dublin City University

Abstract - As Large Language Models (LLMs) are increasingly deployed as autonomous coding assistants and system administrators, their ability to consistently refuse harmful Operating System (OS) and cybersecurity commands is critical. While models are trained with safety alignment techniques like Reinforcement Learning from Human Feedback (RLHF), recent studies suggest this alignment is superficial and highly brittle. This review examines the current landscape of LLM safety, focusing on the specific vulnerability of safety filters to paraphrasing and linguistic variation. We analyze existing benchmarks like *Do-Not-Answer* and *CySecBench*, which provide the foundational taxonomies for testing OS-level harm but suffer from static evaluation protocols that fail to capture real-world usage. Furthermore, we review methodologies for data augmentation, establishing that LLM-generated paraphrasing is a scientifically valid technique for stress-testing safety boundaries. The literature indicates a significant gap: while complex optimization attacks exist there is a lack of focused research quantifying the "inconsistency rate" of models-the frequency with which a model refuses a harmful command in one phrasing while complying with a semantic equivalent and necessitating the development of "automated frameworks" that utilize adversarial paraphrasing to quantify model inconsistency.

Keywords— Large Language Models (LLMs), Safety Alignment, Adversarial Robustness, Jailbreaking, Prompt Injection, Operating System (OS) Security, Reinforcement Learning from Human Feedback (RLHF), Cybersecurity.

I. INTRODUCTION

In modern days, LLMs have become the backbone of many engineering workflows and use cases containing sensitive information. As these models become popular and widely integrated into society, they must comply with safety measures and not be easily exploitable. Evaluating LLMs is critical because models can generate harmful responses and content, including destructive codebases and content enabling malicious activity.

Over the years, significant efforts have been made to evaluate the safety mechanisms of LLMs, and several vulnerabilities have been documented. Jailbreaking attacks are designed to bypass the safety guardrails by making simple modifications in the prompts, such as adding suffixes (a short sequence of tokens appended to the end of any prompt) for the LLM to yield to harmful prompts [11]. Similarly, paraphrasing, which rephrases the harmful requests, has also been effective. These failures showcase the limitations of current safety mechanisms, which often rely on pattern matching and surface-level text features rather than the semantic understanding of the user's intent.

Although recent years have seen substantial growth in LLM safety evaluation and improvements, a review of 144 datasets for LLM safety has been published between 2018 and 2024, which reveals that there is a clear lack of non-English datasets and naturalistic safety evaluation, and current practices are highly idiosyncratic [3]. Moreover, the current literature relies heavily on "static" benchmarking. Static evaluation involves testing a model against a fixed, immutable set of prompt strings (e.g., the exact rows in CySecBench [1]). If the model refuses that specific string, it is deemed safe. This approach fails to account for the infinite variability of natural language. In contrast, this review argues for "dynamic" evaluation, a framework where the test inputs are not fixed but are fluidly generated variants. A dynamic assessment does not ask "Did the model refuse prompt X?"; it asks, "How consistent is the model's refusal across 'N' semantic variations of prompt X?" This shift from static binary testing to dynamic consistency testing is essential to quantify the true brittleness of safety alignment. Also, the study focuses on a critical yet underexplored dimension of LLM safety, which is to evaluate LLMs against paraphrased instructions. Existing research is mostly focused on addressing social harms, such as hate speech and dangerous content. OS safety has real-world consequences, which include system compromise, data exfiltration, and unauthorized access. LLMs that yield dangerous content concerning OS safety require dedicated investigation.

II. REVIEW OF RELATED WORK

A. The Evolution from Static to Dynamic Benchmarks

Safety alignments are put in place to keep LLM behaviors in line with human ethics [8]. Most LLMs are trained on datasets scraped from the internet, which could contain objectionable content, biases, and harmful information [11]. The evaluation of model safety has traditionally relied on static datasets. Wang et al. [2] and Wahréus et al. [1] established the necessary taxonomies for "Destructive OS Behavior," providing standardized baselines for what models

should refuse. LLM safety alignments are implemented post-training to ensure that the model refuses to generate harmful content. Instruction tuning, also known as Supervised Fine-Tuning, trains the model on supervised data consisting of high-quality prompt-response demonstrations [13][15] while Reinforcement Learning from Human Feedback (RLHF) utilises human preference data to train a reward model. These techniques or standard guides LLMs to produce outputs that annotators judge as safe and helpful.

In [15], the survey done on LLM safety proposed a three-layer hierarchical framework of LLM safeguards. The innermost layer was reinforced during training via safety alignment algorithms and specialised datasets, which showcased refusal of harmful queries while the middle layer used system prompts and in- context learning to guide the model behaviour via self-checks. An outermost layer was integrated, which deployed external modules to detect and filter malicious inputs and restrict harmful outputs.

However, these fixed datasets fail to reflect the "messiness" of real-world interactions. Röttger et al. [3] critique these static approaches for lacking ecological validity. In response, the field is shifting toward dynamic benchmarking frameworks. Shen et al. [4] analyzed "in-the-wild" jailbreaks (JailbreakHub), demonstrating that automated, adaptive testing uncovers significantly more vulnerabilities than static counterparts. By iteratively generating test cases based on previous model failures, dynamic frameworks can simulate the adversarial behavior of real users, revealing gaps that static leaderboards like JailbreakBench [5] often miss.

Limitations— Current alignment techniques are often called superficial [8]. Models are typically aligned using natural language, which leaves them vulnerable to instructions that have slight variations in input, such as instructions embedded in code or not specific harmful words. The model does not generalise effectively in these cases, and current research shows that alignment can be easily compromised by fine tuning model or paraphrasing the prompt instructions. This reveals an alarming offence-defense asymmetry

B. Defining Ground Truth: OS and Cybersecurity Harm

To evaluate safety effectively, researchers must first define what constitutes a "harmful" command in a technical context. Early safety work relied on broad, open-ended datasets that mixed hate speech with violence, which made it impossible to isolate specific technical risks [11]. Recent advancements have addressed this by establishing domain-specific taxonomies that separate "social harm" (e.g., toxicity) from "functional harm" (e.g., malware).

Wang et al. [2] introduced the Do-Not-Answer dataset, which established a crucial taxonomy specifically categorizing "Destructive Behavior in Operating Systems" as a distinct risk profile. This provides the necessary ground truth for the types of commands a model should refuse, distinguishing between a benign query (e.g., "how to delete a file") and a malicious one (e.g., "how to wipe a root directory"). Building on this, Wahréus et al. [1] developed CySecBench, a dataset focused entirely on cybersecurity and OS exploits. They argue that general safety datasets lack the technical nuance required to trigger specific failure modes in coding-capable models, which often prioritize being "helpful" over being safe when interpreting technical jargon.

Limitations— Despite these advancements, a critical resource gap remains there is currently no single, comprehensive dataset that isolates "Destructive OS Behavior" with sufficient scale or linguistic variability for robust testing. To address this scarcity, our practicum does not rely on a single benchmark. Instead, we curate a custom composite dataset by extracting validated "seed prompts" from these high-quality sources ([1], [2]). We then address the lack of variability by systematically transforming these seeds into an expanded dataset using the specific paraphrasing methodologies defined in the section below.

C. From Ground Truth to Failure Modes: How Linguistic Variability Breaks Safety Alignment

The theoretical justification for testing paraphrased inputs lies in the internal mechanics of safety failure. Current literature classifies paraphrasing into three distinct categories, each exploiting a specific brittleness in model alignment:

1. *Lexical Modification*

Research by Salles et al. [10] provides critical insight into why simple word swaps are effective. They demonstrate that safety mechanisms often rely on "spurious correlations," where the model learns to associate specific tokens (trigger words like "hack," "malware," or "kill") with a refusal response. However, the model does not necessarily learn the underlying concept of harm. Consequently, if an attacker removes these "trigger words" via lexical paraphrasing while keeping the semantic meaning identical, the model's safety activation fails. Shen et al. [4] validated this in their "in-the-wild" study of 300k+ real-world attacks, finding that actual users rarely employ complex code-based exploits. Instead, they predominantly use paraphrasing and synonyms to bypass these token-level filters.

2. *Logic-Based Rephrasing*

Beyond simple synonym replacement, models are vulnerable to logical inversions. [9] characterized the "Reasoning Backwards" attack, showing that supposedly robust models like GPT-4o can be bypassed by simply instructing them to "output the inverse" of a refusal or reason in reverse order. This finding highlights a failure in the model's ability to maintain safety constraints when the focus of the task is shifted from direct generation to logical deduction.

3. *Back-Translation and Style Transfer*

The third category of paraphrasing involves "linguistic scrubbing" through Back-Translation and Style Transfer. Unlike contextual attacks that require long chains of history, this method generates self-contained, direct prompts that bypass the filter by altering the surface form of the instruction while preserving its semantic core. This process effectively strips away the specific words that safety filters overfit. By forcing the prompt through a different linguistic latent space, the specific "trigger tokens" (like "keylogger" or "exploit") are often replaced with synonyms or structural variations that the safety filter does not recognize, even though the harmful intent remains identical.

Limitations—Despite these type of Paraphrasing, current literature often focuses on finding a single "universal" attack string, such as the optimization-based GCG suffix proposed

by Zou et al. [11]. While scientifically interesting, these attacks are computationally expensive and rare in casual usage. There is a lack of comparative analysis quantifying how vulnerable leading models are to the simple rephrasing techniques described above, which represent a much higher risk for wide-scale deployment.

D. Current Automated Frameworks and their Limitations

To move beyond static datasets, the research community has begun developing automated frameworks for "Red Teaming." However, a significant gap remains between general-purpose attack tools and the specialized needs of OS-level security testing.

Current Automated Frameworks scale adversarial testing beyond manual effort; the research community has developed Automated Red Teaming frameworks. These generally fall into two categories: optimization-based and agent-based generation.

1. Optimization Frameworks: The most prominent example is the Greedy Coordinate Gradient (GCG) algorithm introduced by Zou et al. [11]. This method treats prompt generation as a mathematical optimization problem, automatically appending adversarial suffixes (random character strings) to a query until the target model's refusal probability drops to zero.
2. Agent-Based Red Teaming: Perez et al. [7] established the standard for "LM-vs-LM" architectures. In this framework, a "Red LM" is prompted to generate thousands of conversational variants to probe a "Target LLM." Similarly, Shen et al. [4] developed JailbreakHub, a dynamic framework that aggregates real-world prompt strategies to continuously test models against evolving "in-the-wild" user tactics.

Limitations— The "Judge" Problem and Ecological Validity Despite their scalability, these frameworks suffer from critical methodological flaws when applied to OS-level security:

The "Judge" Failure (Refusal Propagation): A fundamental limitation of automated frameworks is the reliance on "LLM-as-a-Judge" to score attacks. Perez et al. [7] and Qi et al. [8] highlight that when an attack is successful, and the target model generates a harmful malware script, the "Judge" model often refuses to read or grade the output because its own safety filters trigger. This leads to high False Negative rates, where valid jailbreaks are incorrectly marked as refusals because the automated judge is too "safe" to evaluate the harm.

Lack of Ecological Validity: As noted by Wei et al. and Jain et al., optimization methods like GCG generate "gibberish" inputs (e.g., !x#zka%) that are easily detected by perplexity filters. They do not represent semantic or technical attacks used by real users.

Domain Blindness: General red-teaming tools lack the specific knowledge defined in CySecBench [1]. They treat all refusals equally, failing to distinguish between a model refusing a "rude joke" versus a model refusing a "root privilege escalation," rendering them ineffective for high-stakes cybersecurity evaluation.

III. Comparative Analysis

1. Foundational Datasets and Benchmarks: From Static Taxonomies to a Dynamic Evaluation Need

The existing literature defines a substantive, if incomplete, foundation for defining and categorising harm in large language model (LLM) outputs. Do-Not-Answer [2] and CySecBench [1] are pivotal contributions in this regard, providing structured categories of harmful behaviour specifically “Destructive OS Behaviour” and “Cybersecurity Exploits” that give safety researchers a shared vocabulary for threat classification. Similarly, JailbreakBench [5] formalised the adversarial evaluation landscape by introducing a standardised leaderboard tracking model performance against known attack strings, providing a reproducible and comparable baseline across model versions.

Taken together, Do-Not-Answer, CySecBench, and JailbreakBench provide the harm taxonomies and baseline methodology upon which this practicum builds but none measure what this practicum is designed to measure: the rate at which a model produces inconsistent safety decisions across linguistically varied but semantically equivalent adversarial inputs exploiting OS vulnerabilities. This gap motivates the core design of the proposed Hybrid Adversarial Evaluation Framework.

2. Mechanisms of Brittleness under Linguistic Variability: Why Paraphrasing is the Primary Threat Vector

The most significant finding emerging from a synthesis of the literature is that the dominant failure mode for aligned LLMs is not a sophisticate mathematical optimisation, but rather simple semantic restructuring. This has direct implications for how safety evaluations should be designed.

The Greedy Coordinate Gradient (GCG) method introduced by Zou et al. [11] represents the state-of-the-art in optimisation-based adversarial attacks, generating adversarial suffixes through iterative gradient search to produce inputs that reliably elicit harmful outputs. GCG is scientifically significant it demonstrates that aligned models possess exploitable internal decision boundaries, but it produces inputs that are syntactically unintelligible: concatenations of semantically meaningless tokens that no genuine user would produce organically. This limits GCG’s practical threat relevance: real-world adversarial inputs are not random token strings; they are natural language utterances crafted by users who understand, at least intuitively, how to reframe a request to avoid detection.

This distinction is not merely academic. Saiem et al. [6] demonstrate that embedding harmful intent within a “sequential chain” of otherwise benign text a technique requiring no gradient computation, only linguistic creativity is frequently just as effective as GCG at eliciting policy violations. Critically, this approach is intelligible, naturalistic, and scalable: any motivated user can demonstrate it without technical expertise. The implication is that the threat landscape is dominated not by adversarial machine learning but by adversarial linguistic, the strategic paraphrasing, contextual embedding, and semantic disguising of harmful intent within natural language.

Qi et al. [8] provide a further and mechanistically distinct piece of evidence for this brittleness. Their finding that even

minimal fine-tuning can catastrophically degrade safety alignment suggests that the representations underlying a model's safety behaviour are not robustly distributed across its weights but are instead localised and fragile what might be termed "shallow" alignment. While this finding concerns fine-tuning rather than linguistic perturbation directly, it is conceptually consistent with the paraphrase brittleness hypothesis: if safety alignment can be disrupted by a small number of gradient updates during fine-tuning, the same underlying fragility may render it susceptible to inputs that activate alternative representational pathways through linguistic variation alone. In other words, fine-tuning and paraphrasing may both exploit the same fundamental characteristic that safety alignment is not robustly enforced across the full distribution of semantically equivalent inputs.

Together, these studies support the central hypothesis of this practicum: that linguistic attacks are the primary real-world failure mode for aligned LLMs, because they are semantic, intelligible, difficult to filter by keyword, and unlike GCG's token-level perturbations indistinguishable from legitimate user queries in automated evaluation systems. This diagnosis demands an evaluation methodology capable of detecting inconsistency at the semantic level, not merely failure on a fixed set of known attack strings.

3. Methodological Justification of the Hybrid Adversarial Evaluation Framework

The limitations identified in the foundational datasets and the brittleness literature jointly motivate the two-component design of the proposed framework: an automated adversarial generation engine and a Human-in-the-Loop (HITL) evaluation protocol. These components are not independently justified additions; they are complementary responses to a single structural problem in existing methodology.

3.1 Validating the Generative Engine (Automated Red Teaming)

The shift from manual to automated adversarial prompt generation is strongly supported by Perez et al. [7], who demonstrate that automated red-teaming agents systematically uncover failure modes that human testers routinely miss not because human testers are unskilled, but because manual red-teaming is subject to cognitive bias, limited prompt diversity, and fatigue. Automated generation addresses these constraints by enabling systematic production of large, diverse prompt sets across multiple paraphrase strategies, role-play framings, and contextual embeddings of harmful intent. For this practicum specifically, automated generation is essential because the primary object of measurement is the Inconsistency Rate. This requires not a single adversarial prompt per harmful category, but multiple semantically equivalent variants of each prompt. Without automated generation, producing a sufficiently large and varied prompt corpus across the harm categories identified by Do-Not-Answer [2] and CySecBench [1] would be infeasible at scale.

3.2 Validating the Evaluation Protocol (Human-in-the-Loop)

While adversarial generation can and should be automated, evaluation cannot be delegated entirely to language models without introducing systematic measurement error. Qi et al. [8] identify a critical failure mode in LLM-as-Judge paradigms, termed "refusal propagation": when a model is asked to evaluate whether a harmful output has been produced, the evaluator model may itself refuse to process the harmful content not because the output was safe, but because the evaluator is applying its own safety filters to the evaluation task. This produces systematic false negatives, where successful jailbreaks are classified as evaluation failures and excluded from the measurement corpus.

This is not a minor calibration issue; it is a structural validity threat. In fully automated benchmarks such as JailbreakBench [5], refusal propagation creates an upward bias in measured safety performance: models appear safer than they are because the measurement instrument is itself subject to the same filtering behaviour as the target. The HITL layer in this framework directly addresses this threat by interposing human judgment at the classification stage, ensuring that model outputs are assessed against the harm criteria established by Do-Not-Answer [2] and CySecBench [1] without the confounding effect of evaluator-level refusal. The combination of automated generation and HITL evaluation thus constitutes a methodologically coherent response to the two principal weaknesses of existing approaches: the coverage limitation of manual red-teaming, and the measurement validity limitation of fully automated evaluation pipelines.

4. Gap Analysis: Toward an Inconsistency - Centered Evaluation Paradigm

The preceding review reveals a consistent pattern across the literature: existing safety evaluation frameworks are designed to answer a question, "*Does this model refuse this prompt?"* and they are optimised accordingly. Do-Not-Answer [2], CySecBench [1], and JailbreakBench [5] all measure refusal rates against fixed set; GCG [11] measures attack success rates against optimised token sequences; and automated red-teaming approaches [7] measure coverage of the failure mode distribution. What none of these frameworks measure is whether a model makes consistent safety decisions across semantically equivalent inputs.

This distinction is consequential. A model that refuses 95% of adversarial prompts in a fixed benchmark may still exhibit a high *Inconsistency Rate against* the probability that a harmful request, when paraphrased, produces a different safety decision than the original phrasing. An Inconsistency Rate of even 20–30% across harm category represents an exploitable safety failure: any sufficiently motivated adversary who generates linguistic variations of a refused request has a meaningful probability of success without deploying any technical attack tooling. The Inconsistency Rate is therefore a more operationally meaningful metric than refusal rate, as it measures robustness to the adversarial strategy that is dominant in practice: natural language variation.

This practicum diverges from the optimisation-centric paradigm of Zou et al. [11] on both empirical and practical grounds. Empirically, Saiem et al. [6] and the broader naturalised attack literature [4, 10] demonstrate that linguistically restructured inputs are at least as effective as GCG-style attacks in eliciting harmful outputs, without producing the unintelligibility that makes GCG attacks detectable by perplexity-based filters. Practically, linguistic attacks require no gradient access, no model weights, and no technical expertise making them accessible to a far wider adversarial population. The risk profile of linguistic attacks

therefore substantially exceeds that of token-optimisation attacks when assessed against realistic threat models.

Existing safety evaluations thus leave a three-dimensional methodological gap: they lack (1) scalability in adversarial generation across linguistically diverse paraphrase strategies; (2) naturalism in linguistic structure, relying on either human-generated prompts of limited diversity or optimised inputs of limited real-world relevance; and (3) measurement validity, due to the refusal propagation problem in automated evaluation pipelines. The Hybrid Adversarial Evaluation Framework proposed in this practicum is designed to close all three dimensions of this gap simultaneously.

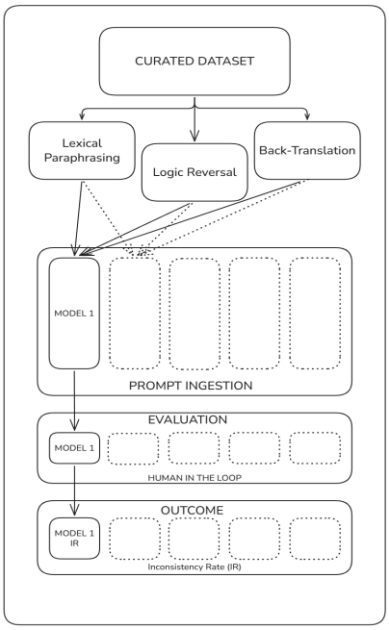
It is worth acknowledging that this framework carries its own limitations. The HITL layer introduces inter-rater variability and scalability constraints: human classification of harmful outputs is subject to annotator disagreement, particularly in borderline cases, and cannot scale indefinitely. Furthermore, the Inconsistency Rate as a metric assumes that semantic equivalence across paraphrased prompts can be reliably established a non-trivial assumption addressed in this practicum through careful prompt design, but one that may be imperfect in practice. These limitations are acknowledged as directions for future work, and do not undermine the foundational case that inconsistency-centred evaluation represents a methodologically necessary advance over current binary refusal-rate measurement.

IV. Discussion

The findings synthesised in this review converge on a single practical imperative: the LLM safety evaluation paradigm must shift from binary refusal testing toward consistency-centred assessment. The Hybrid Adversarial Evaluation Framework proposed in this practicum operationalises this shift through three integrated components, each grounded in the reviewed literature.

Conceptual Framework of the Proposed Practicum

- The following chart maps how our practicum integrates the reviewed literature into a cohesive operational framework:



1) Sourcing Validated Ground Truth.
   - First, harmful seed prompts will be sourced from validated categories in Do-Not-Answer [2] and CySecBench [1]. This ensures that the evaluation targets objectively harmful OS and cybersecurity commands (e.g., "delete root directory") rather than subjective toxicity (e.g., "insults").
2) The Automated Adversarial Framework
   - Second, drawing on established work in LLM-based Data Augmentation [12] and Automated Red Teaming [7], we will develop a "Testing Framework". We ingest static seed prompts into paraphrasers, such as Parrot or Pegasus etc., and perform real-time expansion under paraphrasing categories.
3) Final Metric (The Outcome)
   - Goal: Compute the "Inconsistency Rate" (Robustness Curve) rather than a static leaderboard score.
   - By generating and releasing an augmented dataset and the framework of paraphrased OS-level attacks, this work contributes toward ecologically valid, safety benchmarks that better reflect the "messiness" of real human language.

V. Conclusion

The literature concluded in this review leads to a clear and concerning conclusion: the safety alignment of modern Large Language Models (LLMs) remains fundamentally brittle. While Reinforcement Learning from Human Feedback (RLHF) has reduced explicit high-toxicity outputs [15], it does not provide reliable protection against semantic manipulation. Evidence from [10] and [9] indicates that many safety filters behave less like true intent classifiers and more like advanced keyword detectors. As a result, refusal behaviour is often triggered by specific tokens rather than a genuine understanding of harmful intent. When these trigger words are removed through paraphrasing, the safety mechanism frequently collapses, demonstrating a persistent generalization mismatch across linguistic variations [4].

Another key finding is the disconnect between current evaluation practices and real-world adversarial behaviour. Benchmarks such as Do-Not-Answer [2] and CySecBench [1] provide essential categories for defining OS and cybersecurity harm, yet their prompt structures fail to capture the variable nature of real attacks. Studies on in-the-wild jailbreaks [4] and sequential prompt manipulation [6] show that adversaries rely on language restructuring and slight shifts in input rather than fixed attack strings. Hence, evaluating models only on immutable prompts from datasets like AdvBench [11] or JailbreakBench [5] likely overestimates actual robustness. A model that refuses one phrasing but complies with a semantic equivalent cannot be considered truly safe, only inconsistent.

The move toward variable assessment suggests a future where safety is not measured by a single score, but by a "robustness curve." Future work must focus on "Semantic Safety Filters" that evaluate the intent of a user's query regardless of the phrasing used. Furthermore, the framework proposed here can be extended beyond OS commands to other high-risk domains, serving as a generalizable tool for "red-teaming" AI systems before deployment.

REFERENCES

[1] J. Wahréus, A. M. Hussain, and P. Papadimitratos, "CySecBench: Generative AI-based CyberSecurity-focused Prompt Dataset for Benchmarking Large Language Models," Jan. 02, 2025, *arXiv*: arXiv:2501.01335. doi: 10.48550/arXiv.2501.01335.

[2] Y. Wang, H. Li, X. Han, P. Nakov, and T. Baldwin, "Do-Not-Answer: Evaluating Safeguards in LLMs".. Do-Not-Answer: Evaluating Safeguards in LLMs.

[3] P. Röttger, F. Pernisi, B. Vidgen, and D. Hovy, "SafetyPrompts: A Systematic Review of Open Datasets for Evaluating and Improving Large Language Model Safety", *AAAI*, vol. 39, no. 26, pp. 27617-27627, Apr. 2025. doi: 10.1609/aaai.v39i26.34975 .

[4] X. Shen, Z. Chen, M. Backes, Y. Shen, and Y. Zhang, "'Do Anything Now': Characterizing and Evaluating In-The-Wild Jailbreak Prompts on Large Language Models," in *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security*, Salt Lake City UT USA: ACM, Dec. 2024, pp. 1671–1685. doi: 10.1145/3658644.3670388

[5] P. Chao *et al.*, "JailbreakBench: An Open Robustness Benchmark for Jailbreaking Large Language Models," Oct. 31, 2024, *arXiv*: arXiv:2404.01318. doi: 10.48550/arXiv.2404.01318.

[6] B. A. Saiem, S. H. Shanto, R. Ahsan, and R. U. Rashid, "SequentialBreak: Large Language Models Can be Fooled by Embedding Jailbreak Prompts into Sequential Prompt Chains". SequentialBreak: Large Language Models Can be Fooled by Embedding Jailbreak Prompts into Sequential Prompt Chains.

[7] E. Perez *et al.*, "Red Teaming Language Models with Language Models," in *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*, Abu Dhabi, United Arab Emirates: Association for Computational Linguistics, 2022, pp. 3419–3448. doi: 10.18653/v1/2022.emnlp-main.225.

[8] X. Qi *et al.*, "Fine-tuning Aligned Language Models Compromises Safety, Even When Users Do Not Intend To!," Oct. 05, 2023, *arXiv*: arXiv:2310.03693. doi: 10.48550/arXiv.2310.03693.

[9] M. Bombieri, S. Paolo Ponzetto, and M. Rospocher, "The Dangerous Effects of a Frustratingly Easy LLMs Jailbreak Attack," *IEEE Access*, vol. 13, pp. 126418–126431, 2025, doi: 10.1109/ACCESS.2025.3589112.

[10] M. M. Salles, P. Goyal, P. Sekhsaria, H. Huang, and R. Balestriero, "Paraphrasing Away Malicious Tokens: Improving LLM Finetuning Safety by Filtering Spurious Correlation".

[11] A. Zou, Z. Wang, N. Carlini, M. Nasr, J. Z. Kolter, and M. Fredrikson, "Universal and Transferable Adversarial Attacks on Aligned Language Models," Dec. 20, 2023, *arXiv*: arXiv:2307.15043. doi: 10.48550/arXiv.2307.15043.

[12] B. Ding *et al.*, "Data Augmentation using LLMs: Data Perspectives, Learning Paradigms and Challenges," in *Findings of the Association for Computational Linguistics ACL 2024*, Bangkok, Thailand and virtual meeting: Association for Computational Linguistics, 2024, pp. 1679–1705. doi: 10.18653/v1/2024.findings-acl.97.

[13] W. Cheng, K. Sun, X. Zhang, and W. Wang, "Security Attacks on LLM-based Code Completion Tools", *AAAI*, vol. 39, no. 22, pp. 23669-23677, Apr. 2025. Doi: 10.1609/aaai.v39i22.34537

[14] M. Nadăş, L. Dioşan, and A. Tomescu, "Synthetic Data Generation Using Large Language Models: Advances in Text and Code," *IEEE Access*, vol. 13, pp. 134615–134633, 2025, doi: 10.1109/ACCESS.2025.3589503.

[15] Z. Dong, Z. Zhou, C. Yang, J. Shao, and Y. Qiao, "Attacks, Defenses and Evaluations for LLM Conversation Safety: A Survey," in *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, Mexico City, Mexico: Association for Computational Linguistics, 2024, pp. 6734–6747. doi: 10.18653/v1/2024.naacl-long.375.